

EEE3096S Practical 2

Memory-Mapped I/O and Hardware SPI

Building and Debugging an EEPROM Driver

Department of Electrical Engineering
University of Cape Town
2026

1 Overview

In the previous practicals you used the STM32 through a mixture of HAL functions, timers, direct measurements, C, and Assembly. In this practical you move one layer closer to the hardware.

You will configure and operate a communication peripheral by accessing its memory-mapped registers directly. You will then prove that the configuration is correct by observing the electrical signals generated by the microcontroller.

The practical is centred on the SPI EEPROM already fitted to the UCT Development Board. You do not need an additional external peripheral.

The board contains a CAT25040LI-G serial EEPROM connected to the STM32 through the following signals:

Signal	STM32 pin	EEPROM function
CS	PB12	Chip Select
MISO	PB13	Serial Output from EEPROM
MOSI	PB14	Serial Input to EEPROM
SCK	PB15	Serial Clock

The practical has six tasks. They build on one another:

1. Access a GPIO using memory-mapped registers.
2. Configure the STM32 hardware SPI peripheral without HAL SPI functions.
3. Verify the generated SPI waveform using the oscilloscope.
4. Build a working driver for the onboard EEPROM.
5. Diagnose and repair a deliberately broken SPI configuration.
6. Convert the EEPROM transaction into a non-blocking finite state machine.

This is not a bit-banging practical.

The assessed SPI communication must be generated by the STM32 hardware SPI peripheral. You may not reproduce SPI by manually toggling GPIO pins.

A later course activity deals with implementing communication protocols using ordinary GPIO. Practical 2 instead focuses on understanding how the microcontroller's actual hardware peripheral is configured and controlled.

By the end of this practical, you should be able to move from a statement such as

“send one byte using SPI”

to an explanation of:

clock tree → peripheral registers → alternate functions → SPI hardware → electrical waveform → peripheral response.

2 Outcomes and Knowledge Areas

After completing this practical, you should be able to:

- Explain how memory-mapped I/O allows the processor to control peripherals.
- Explain why hardware-register declarations require `volatile`.
- Locate peripheral base addresses and register definitions in the STM32 documentation.
- Configure GPIO mode and alternate-function registers directly.
- Configure the STM32 hardware SPI peripheral using its control registers.
- Calculate the SPI clock frequency from the system and peripheral clocks.
- Interpret SPI clock polarity, clock phase, and bit order from an oscilloscope waveform.
- Use a peripheral datasheet to construct valid command transactions.
- Write and read non-volatile data using the onboard EEPROM.
- Diagnose communication failures using register values and measured electrical signals.
- Implement a deterministic finite state machine around a hardware communication peripheral.
- Distinguish blocking peripheral code from non-blocking embedded software.

3 Assessment and Deliverables

This practical is assessed primarily through the live demonstration.

Task	Live Demo	Written Evidence
Task 1: Memory-mapped GPIO	8	2
Task 2: Hardware SPI configuration	12	3
Task 3: SPI waveform verification	12	3
Task 4: EEPROM driver	16	4
Task 5: Fault diagnosis	16	4
Task 6: Non-blocking FSM	16	4
Total	80	20

The written submission is therefore a short engineering record, not a long formal report.

Your practical mark depends heavily on your ability to reproduce, modify, and explain your work at the bench.

A result shown in the PDF but which you cannot reproduce or explain during the live demonstration does not substitute for the demonstration marks.
You are responsible for understanding every register write and every significant line of your submitted implementation.

You must:

- Push your complete source code to your group GitHub repository.
- Submit one short PDF to Gradescope.
- Demonstrate the practical in person in the White Lab.
- Bring the development board and a working build environment to the demo.

Do not submit video files.

Do not submit links to videos.

Videos are not used for the assessment of Practical 2.

4 Getting Started

Complete these steps before writing code.

1. Clone the course Git repository again.

Do not rely on the local copy that you used for Practical 1A or Practical 1B. Clone a fresh copy so that you have the current Practical 2 starter files.

```
git clone https://github.com/EEE3096S-UCT/EEE3096S-2026.git
```

Then enter the Practical 2 directory:

```
cd EEE3096S-2026/Practical_2/Practical_2A
```

2. Read the Practical 2 README before opening the project.

The file

```
Practical_2A/README.md
```

contains practical-specific information about the starter project, supplied files, the deliberate fault case, and any updates made after publication of this document.

You are responsible for reading it.

3. Open STM32CubeIDE.

Import the supplied project using:

File → Import → Existing Code as Makefile Project

Select the Practical 2 project directory and use the MCU ARM GCC toolchain.

4. Inspect the supplied source before changing anything.

Identify: the startup code, the system clock configuration, the supplied helper functions, the TODO locations, the fault-case source identified in the README.

5. If an `.ioc` file is supplied, you may inspect it, but do not use CubeMX to configure the assessed SPI peripheral.

Do not regenerate the project unless the README explicitly instructs you to do so.

5 Equipment and Reference Documents

You require: One UCT Development Board fitted with the STM32F051C8. The onboard CAT25040LI-G SPI EEPROM. A two-channel digital oscilloscope. Oscilloscope probes. A PC running STM32CubeIDE or the supplied GNU ARM toolchain. The onboard ST-Link debugger/programmer.

You will need to consult the following documents: STM32F051x4/x6/x8 datasheet. RM0091 STM32F0 reference manual. CAT25040LI-G EEPROM datasheet. UCT Development Board schematic or pinout reference. The Practical 2 README.md. Part of the assessment is your ability to find the required information in these documents.

Where a task asks for a register field, peripheral address, alternate function, command value, timing requirement, or status bit, do not guess it.

6 Rules for the Assessed Implementation

The following rules apply to Tasks 1–6.

6.1 Permitted

You may use: the STM32 CMSIS/device header files, symbolic peripheral register names such as `GPIOx->MODER`, direct pointer access to documented peripheral addresses, your own peripheral-register structures, timers or SysTick for scheduling, the debugger, LEDs and buttons for diagnostic output, short helper functions that you wrote yourself.

6.2 Not permitted for the assessed SPI interface

You may not use: `HAL_SPI_Init()`, `HAL_SPI_Transmit()`, `HAL_SPI_Receive()`, `HAL_SPI_TransmitReceive()`, equivalent LL SPI transfer functions, CubeMX-generated SPI configuration as the solution, a pre-written EEPROM library, GPIO bit-banging to replace the hardware SPI peripheral.

You may use `HAL_GetTick()` purely as a time source in Task 6 if it is already available in the supplied project. It may not perform or replace the assessed SPI communication.

The purpose is not to prove that a library can communicate with the EEPROM.
The purpose is to prove that **you can configure the hardware peripheral, understand the transaction it generates, and diagnose it when it fails.**

7 Board Preparation and Safety

Before starting:

1. Inspect the P2 loopback header.
Remove any loopback jumpers unless a task or the Practical 2 README explicitly instructs you to install one.
PB13, PB14, and PB15 are connected both to the EEPROM and to the P2 header. Incorrectly configuring a looped-back GPIO as an output can cause two outputs to drive against one another.
2. Confirm that PB12, PB13, PB14, and PB15 are the pins connected to the EEPROM on your board.
3. Connect oscilloscope ground only to the board ground.
4. Do not probe or connect signals blindly. Confirm the pin and signal before attaching the probe.

5. Do not repeatedly write to the EEPROM inside an uncontrolled infinite loop. EEPROM memory has finite write endurance. Writes in this practical should occur only when deliberately requested.

8 Background

8.1 Memory-Mapped I/O

The STM32 uses memory-mapped I/O. Peripheral registers occupy addresses in the processor's memory address space.

A register can therefore be accessed using an ordinary C pointer.

Conceptually:

```
1 volatile uint32_t *register_address =  
2     (volatile uint32_t *) SOME_ADDRESS;  
3  
4 *register_address = value;
```

The address is not an arbitrary number. It is determined by the STM32 memory map, the peripheral base address, and the register offset.

For a register at offset O from peripheral base address B ,

$$A_{\text{register}} = B + O.$$

You must be able to show where both B and O came from.

8.2 Why volatile Matters

A hardware register may change independently of ordinary program flow.

For example: a status bit may be changed by the SPI hardware, a receive register may change when a new byte arrives, writing a data register may start a peripheral operation. The compiler must therefore perform the required memory access rather than assuming that a previously read value is still valid. This is one reason that peripheral registers are declared `volatile`.

8.3 Hardware SPI

A hardware SPI peripheral generates the serial timing in dedicated hardware.

Software normally: configures the peripheral, writes data to the SPI data register, waits for or reacts to status flags, reads received data. The hardware peripheral then controls the SPI clock and data timing.

You must determine from the STM32 documentation:

- which SPI peripheral is connected to the EEPROM pins,
- which alternate function is required on PB13, PB14, and PB15,
- the clock source of that SPI peripheral,
- the available SPI clock divisors,
- the meaning of CPOL and CPHA,
- how bit order is selected,
- how transmit-ready, receive-ready, and busy conditions are reported.

9 Tasks

Complete the tasks in order.

Each task depends on a result from the previous task.

Task 1: Memory-Mapped GPIO Access

[10 marks]

Your first task is to prove that you can control a peripheral without a HAL GPIO function.

Use **PC13** on the expansion header as your test output.

Configure PC13 as a digital output by directly accessing the required RCC and GPIO registers.

For the first implementation, use explicit `volatile` register pointers and addresses obtained from the STM32 documentation.

Do not use:

```
1 HAL_GPIO_Init(...);  
2 HAL_GPIO_WritePin(...);  
3 HAL_GPIO_TogglePin(...);
```

Generate a slow repeating HIGH/LOW waveform so that it can be observed easily on the oscilloscope. The exact frequency of this waveform is not assessed.

Live demonstration checkpoint

You must: show the waveform on PC13, show the corresponding register writes in your code, derive at least one register address from its base address and offset, explain what would happen if the GPIO peripheral clock were disabled, explain the purpose of `volatile`. The demonstrator may ask you to identify the register changes required to move the output to another free GPIO pin.

Task 2: Configure the Hardware SPI Peripheral

[15 marks]

You will now configure the STM32 hardware SPI peripheral connected to the onboard EEPROM.

The EEPROM uses:

STM32 pin	Signal
PB12	CS
PB13	EEPROM serial output
PB14	EEPROM serial input
PB15	SPI clock

Do not assume which SPI peripheral or alternate-function number is required. Determine these from the STM32 datasheet.

Configure: the required peripheral clock, PB13, PB14, and PB15 in the correct alternate-function mode, PB12 as a manually controlled GPIO chip-select output, SPI controller/master operation, the required clock polarity and clock phase, the required bit order, an appropriate SPI clock divider, the SPI peripheral enable bit.

Start with a nominal SCK frequency of approximately

250 kHz

Calculate the divider from the actual peripheral clock used by your build. Do not simply enter a divider until the waveform “looks right.” At this stage keep PB12 HIGH so that the EEPROM remains deselected.

Record

Provide a concise configuration table containing:

Item	Your value and justification
SPI peripheral instance	
Peripheral clock frequency	
PB13 alternate function	
PB14 alternate function	
PB15 alternate function	
Clock divider	
Predicted SCK frequency	
CPOL	
CPHA	
Bit order	

For every register field, name the STM32 manual section or datasheet table from which you obtained the information.

Live demonstration checkpoint

You must be able to: show the configuration registers in the debugger, explain each SPI configuration field that you changed, calculate the expected SCK frequency, identify the GPIO alternate-function fields used, explain why PB12 is initially held HIGH.

Task 3: Prove the SPI Waveform

[15 marks]

You now have to prove that the hardware peripheral generates the waveform you claim it generates.

Generate your group test byte

Let:

n_1 = last three decimal digits of student 1’s student number

and

n_2 = last three decimal digits of student 2’s student number.

Calculate:

$$B = [(n_1 \oplus n_2) + 0x3D] \bmod 256.$$

If the result is 0x00 or 0xFF, replace it with:

$$B = B \oplus 0x5A.$$

Record the calculation. This value is your group’s test byte for the remainder of the practical.

Generate the transfer

Keep the EEPROM deselected by holding PB12 HIGH.

Write your value B to the SPI data register and allow the **hardware SPI peripheral** to transmit it. Connect: CH1 to SCK, CH2 to MOSI. Capture one complete byte.

Your oscilloscope capture must allow you to determine: SCK period, SCK frequency, clock idle state, number of clock cycles, data-bit sequence, bit order, the transmitted value.

Manually decode the waveform. Do not use the value in your source code as the proof that the transmission is correct. The waveform is the evidence.

Calculate:

$$f_{\text{SCK,measured}} = \frac{1}{T_{\text{SCK,measured}}}.$$

Then calculate the percentage error relative to your predicted SCK frequency:

$$\% \text{error} = \frac{|f_{\text{measured}} - f_{\text{predicted}}|}{f_{\text{predicted}}} \times 100.$$

Live demonstration checkpoint

During the demonstration you must: transmit your group byte, place the oscilloscope cursors yourself, measure SCK, identify the eight data bits, decode the value from the waveform, explain where the sampling edge occurs. The demonstrator may give you a different byte and ask you to predict its MOSI pattern before you run the program.

Task 4: Build an EEPROM Driver

[20 marks]

You will now use the waveform from Task 3 to communicate with the onboard CAT25040LI-G EEPROM.

Read the EEPROM datasheet before implementing the driver.

Do not use a pre-written EEPROM library.

Your driver must implement at least the following operations:

```
1 uint8_t eeprom_read_status(void);  
2 void eeprom_write_enable(void);  
3 void eeprom_write_byte(uint16_t address, uint8_t value);  
4 uint8_t eeprom_read_byte(uint16_t address);
```

The exact command values, transaction order, timing, and status bits must come from the EEPROM datasheet.

Generate your group EEPROM address

Use:

$$A = (n_1 + 3n_2) \bmod 256.$$

Use address A and test byte B from Task 3 for your default test.

Your program must: read the EEPROM status, enable a write, write B to address A , determine from the status register when the write operation has completed, read address A , verify that the returned

value equals B . Do not use an arbitrary fixed delay as proof that the EEPROM write has completed. Use the EEPROM's documented status mechanism.

For Task 4 it is acceptable for your status-polling loop to block. You will remove this blocking behaviour in Task 6.

Visible verification

After a successful read, display the returned byte on the eight user LEDs:

$$\text{LED7} \dots \text{LED0} = b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0.$$

Your read value must also be available as a debugger variable.

Persistence test

Your implementation must provide a way to read the EEPROM without automatically writing to it first.

The demonstrator must therefore be able to: write a value, reset or power-cycle the board, run the read-only path, recover the previously stored value.

Oscilloscope verification

Capture a real EEPROM transaction. Use one capture showing: CH1 connected to CS, CH2 connected to SCK. The capture must show that the correct number of clock pulses occur while the EEPROM is selected.

Live demonstration checkpoint

You must demonstrate: status-register access, write-enable behaviour, one EEPROM write, write-completion detection, read-back, correct LED display, persistence after reset or power cycle. The demonstrator may select another valid address and byte during the demo.

Task 5: Diagnose a Broken SPI Interface

[20 marks]

The Practical 2 repository contains a deliberately faulty configuration. The README identifies how to build or enable the fault case. It does **not** tell you the cause. The program compiles, but the EEPROM transaction does not behave correctly. Your task is to diagnose the fault using engineering evidence.

Do not solve this task by randomly changing SPI register fields until the EEPROM starts responding. You must identify an observable symptom, connect that symptom to the configuration, and explain why the correction fixes it.

Use: the oscilloscope, peripheral registers, GPIO registers, status flags, the STM32 documentation, the EEPROM datasheet.

Your diagnosis must follow this structure: State what waveform or behaviour you expected. Measure what the board actually produced. Identify the first meaningful difference. Identify the register or configuration responsible. Correct the fault. Repeat the measurement, explain why the corrected result now satisfies the protocol or peripheral requirement.

Required evidence

Provide one oscilloscope capture from the faulty case and one from the corrected case. Both captures must use the same relevant measurement where possible so that the difference can be compared directly. Complete the following table in your report:

Expected	Observed	Cause	Correction

Live demonstration checkpoint

You must explain: the symptom, the register responsible, the incorrect field value, the corrected field value, the electrical reason the transaction failed. You may also be given a small tutor-selected configuration change and asked to predict its effect before the program is run.

Task 6: Non-Blocking EEPROM Transaction FSM [20 marks]

Your Task 4 implementation may wait inside loops while the EEPROM is busy.

A real embedded system often needs to continue responding to other events while a peripheral operation is in progress.

Your final task is therefore to restructure the complete transaction as a deterministic finite state machine.

The required high-level operation is:

request → write-enable → write → wait for EEPROM → read-back → verify → result.

Design your own states.

Your state machine must include enough states to distinguish at least: idle, write preparation, write transaction, EEPROM busy/status checking, read transaction, verification, success or failure. You are not required to use these exact names.

Functional requirements

Use the board controls as follows: PA0 starts a complete write/read/verify operation. PA3 aborts the current software transaction and returns the controller to its idle state. PB0–PB7 display the last byte read from EEPROM. PB11, the green status output, indicates a successful verification. PB10, the red status output, indicates a failed verification. During an active transaction, the main loop must remain responsive to PA3.

You may not use:

```
1 HAL_Delay(...);
```

or an equivalent software busy-wait to wait for the EEPROM's internal write cycle. You may use a system tick or timer to determine when the next status check is due. The state machine should perform a small amount of work, update its state, and return control to the main loop.

Conceptually:

```
1 while (1)
2 {
3     read_inputs();
4     update_eeprom_state_machine();
5     update_outputs();
6 }
```

Do not place the complete EEPROM transaction inside one blocking function called from the loop.

State-machine record

Draw your state diagram before completing the implementation. The diagram must show: initial state, inputs or conditions causing transitions, success path, failure path, abort path.

Live demonstration checkpoint

You must: initiate a transaction using PA0, show the state variable changing in the debugger, demonstrate successful write/read verification, display the returned byte on the LEDs, abort an active transaction using PA3, explain why the implementation is non-blocking. The demonstrator may ask you to change the test address or data byte, rebuild, and repeat the transaction.

10 Written Submission

The written submission is a concise record of the work that supports the live demonstration.

Do not write a long formal laboratory report.

Submit one PDF.

10.1 Format

- Maximum length: **8 pages**, excluding the title page and references.
- Write the report in $\text{\LaTeX}$.
- Do not include full source-code listings.
- Your complete code belongs in the Git repository.
- Use numbered figures and tables.

Filename:

`prac_02_stdnum001_stdnum002.pdf`

Upload the PDF to Gradescope before your demonstration and push the final code to your group repository.

10.2 Required Structure

Use six short numbered sections.

Section 1: Memory-Mapped GPIO

Include:

- the relevant peripheral addresses and offsets,
- the bit masks used,
- a short explanation of `volatile`,
- the Task 1 oscilloscope capture.

Section 2: Hardware SPI Configuration

Include:

- the Task 2 configuration table,
- the SPI clock calculation,
- the relevant alternate-function information,
- the manual sections or datasheet tables used.

Section 3: SPI Waveform

Include:

- calculation of your group byte B ,
- SCK/MOSI oscilloscope capture,
- measured SCK frequency,
- percentage frequency error,
- manual decoding of the transmitted byte.

Section 4: EEPROM Driver

Include:

- calculation of address A ,
- a concise transaction sequence for write and read,
- the status mechanism used to detect write completion,
- your write/read result,
- CS/SCK oscilloscope capture.

Section 5: Fault Diagnosis

Include:

- expected-versus-observed diagnosis table,
- faulty waveform,
- corrected waveform,
- register field responsible,
- short explanation of the correction.

Section 6: Non-Blocking FSM

Include:

- your final state-machine diagram,
- a short description of the transition logic,
- explanation of why the implementation is non-blocking,
- one example write/read/verify result.

10.3 References

List the exact reference-manual sections, datasheet tables, and other technical documents used. Use IEEE referencing style.

A reference such as “STM32 datasheet” is not sufficient when the task asks you to identify a specific alternate-function table or register field.